Digital Chip Design Program

RTL Design

Task #08

Farah Alyahya

Rana Alsubaihi

Joud Almshari

Tala Alrsheed

Hoor Alkhedairi

Horiah Alqufaydi

Sandra Alomar

14/7/2026

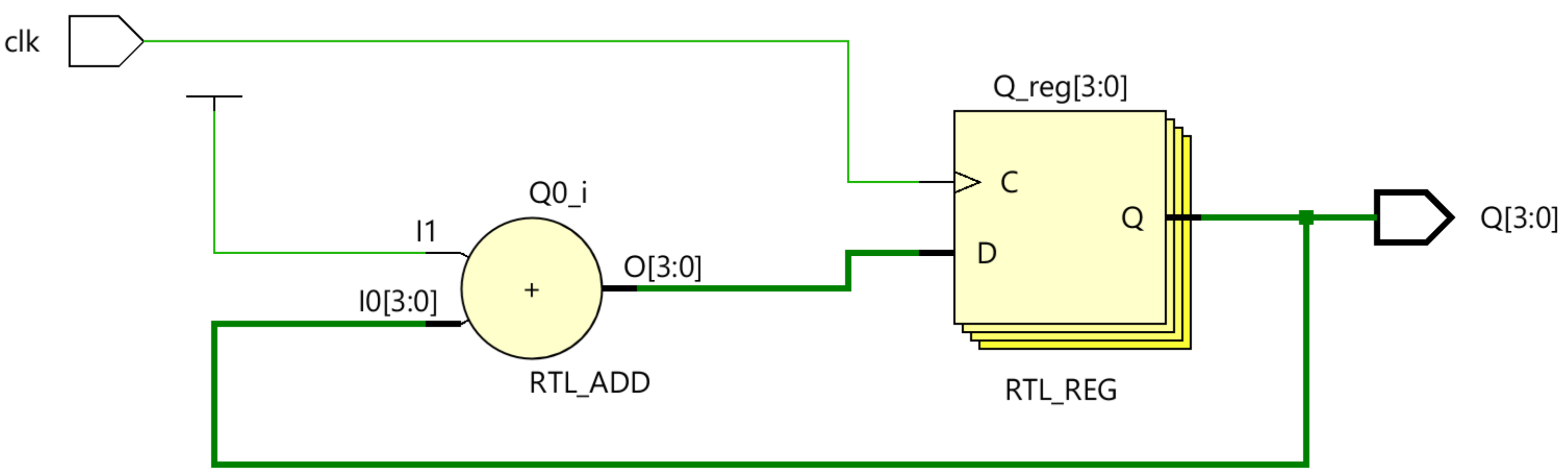
Sequential Datapath Design Part 2

Version 1.0

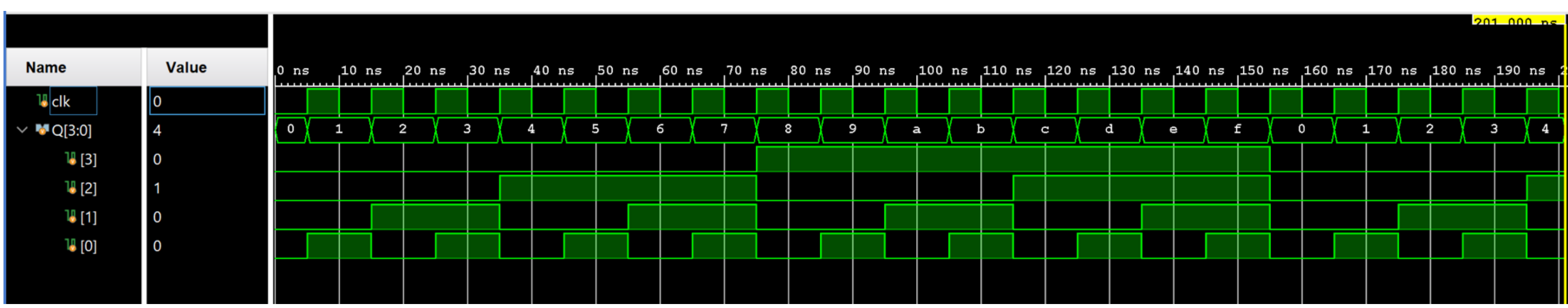
Task #1.1

1.1.1.1

```
module top (
      input logic clk,
      output logic [3:0] Q
);

      always_ff @(posedge clk ) begin

            Q <= Q + 1'b1;
      end
endmodule
```



```
`timescale 1ns / 1ps

module Counter_tb;

    logic clk;
    logic [3:0] Q;

    top uut (
        .clk(clk),
        .Q(Q)
    );

    always begin
        #5 clk = ~clk;
    end

    initial begin
        clk = 0;
        force uut.Q = 4'b0000;
        #1;
        release uut.Q;

        $monitor("Time = %0t | Q = %d (%b)", $time, Q, Q);

        #200;
        $finish;
    end
endmodule
```

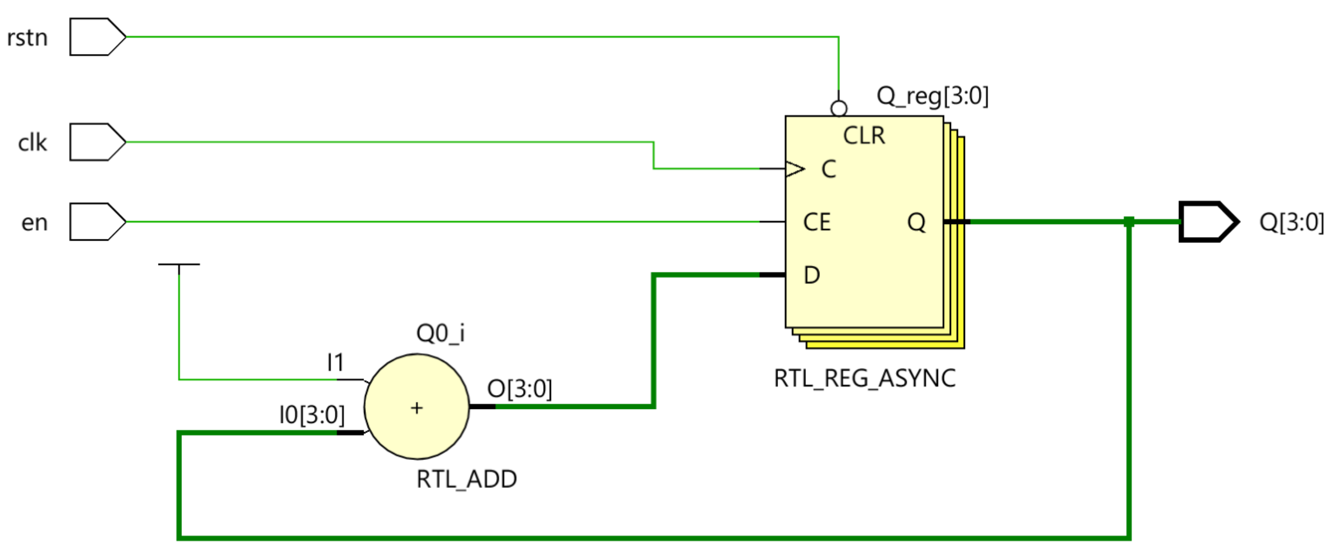



Task #1.2

1.2.1

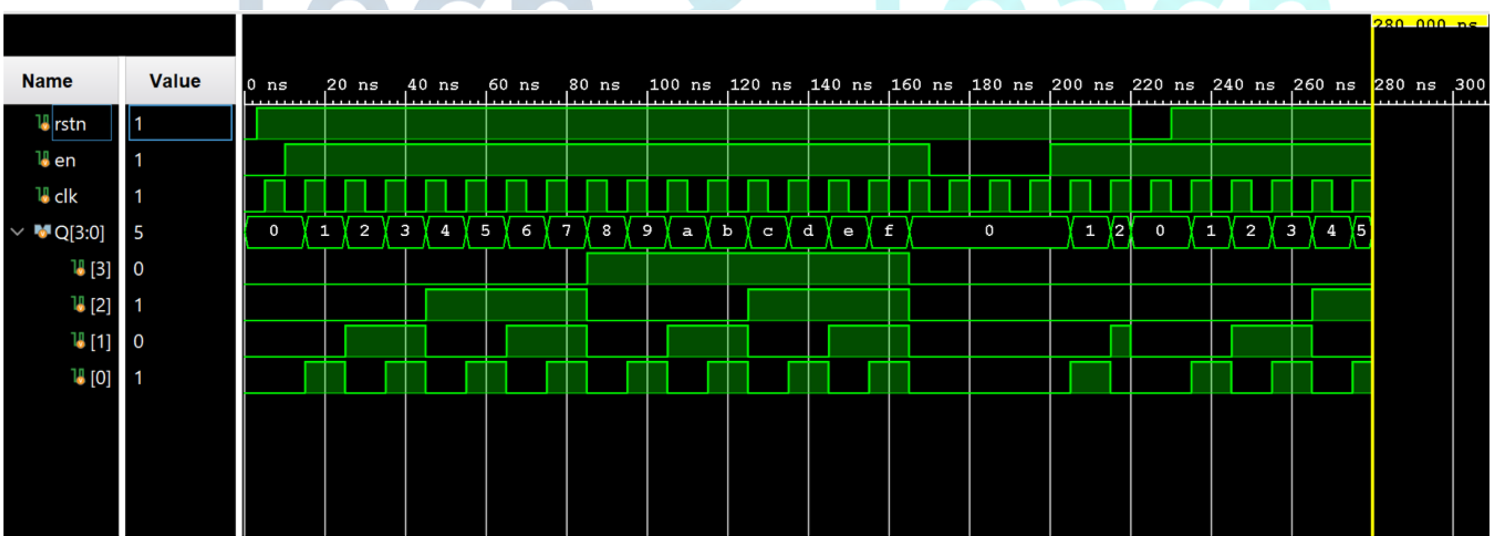
1.2.2

```
module top (
    input logic clk,
    input logic rstn,// Added in Task2 <Asynchronous Active-Low Reset>
    input logic en, //  Added in Task2 <Synchronous Active-High Enable>
    output logic [3:0] Q
);

    always_ff @(posedge clk or negedge rstn ) begin // Updated:negedge of rstn
            if (!rstn)
            Q <= 4'b0000; // Added in Task2 to Clear counter immediately when reset is active
            else if (en) // Updated: Counter only increments when enable is high
            Q <= Q + 1'b1;
    end
endmodule
```



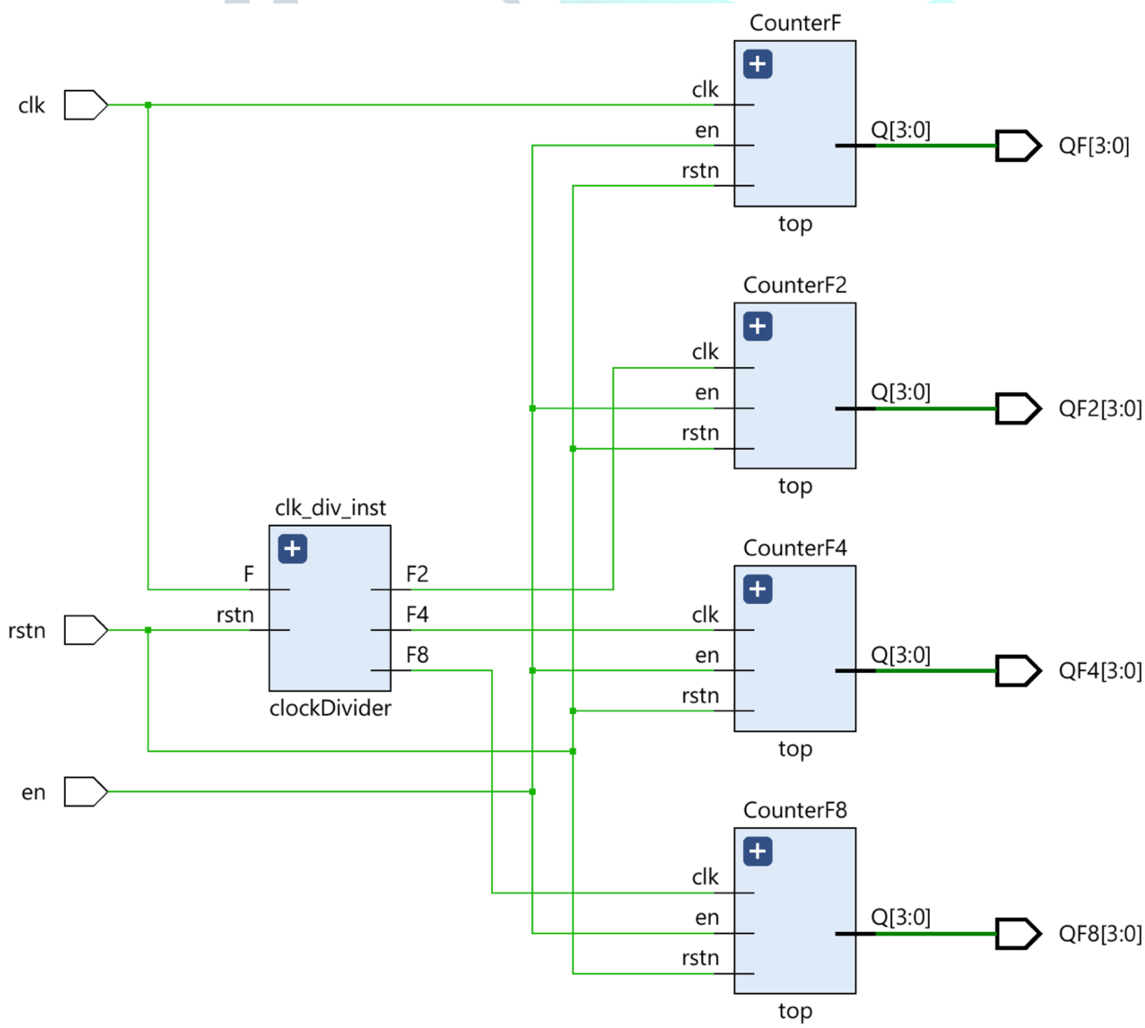
1.2.3

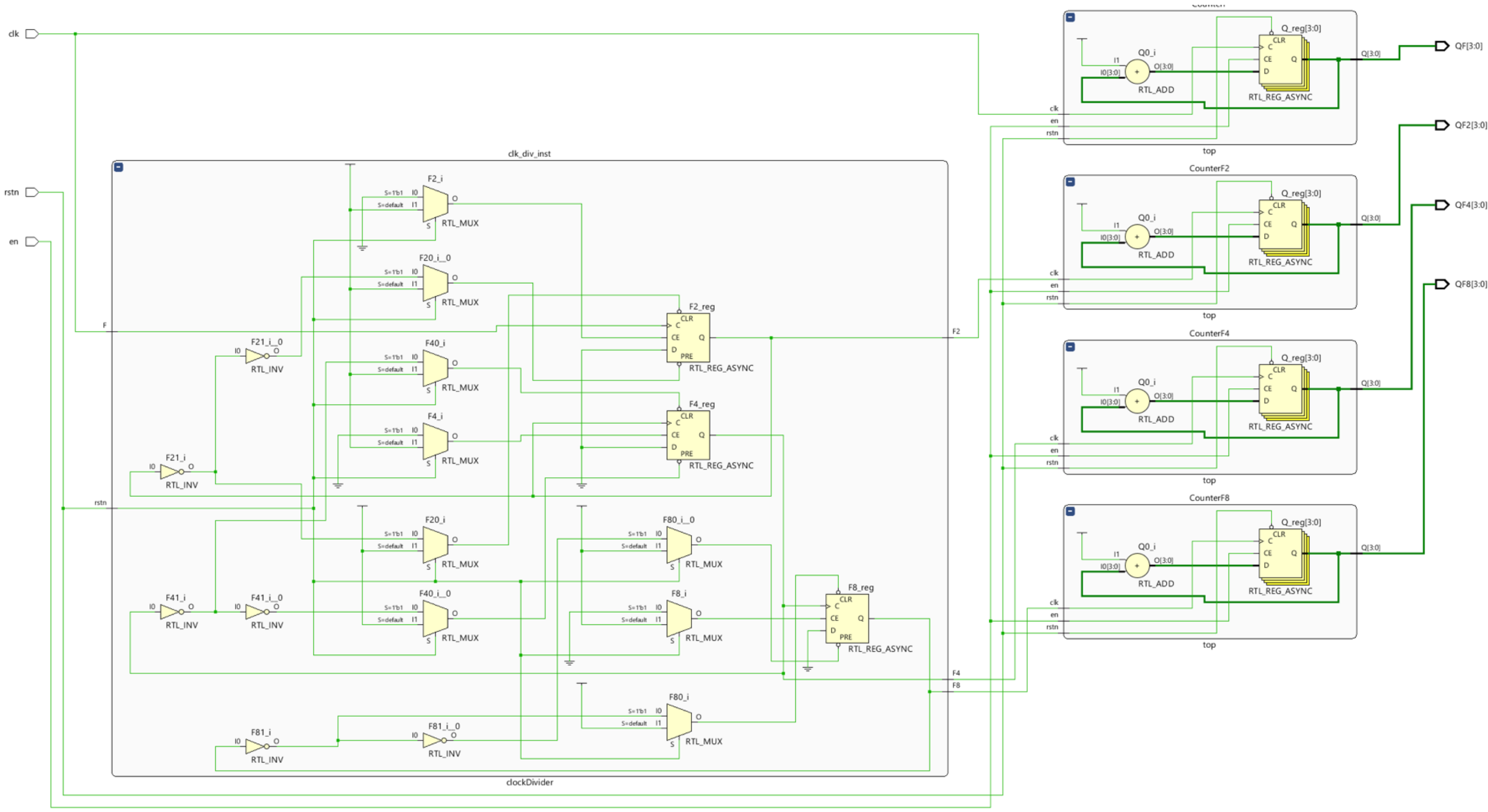
```
`timescale 1ns / 1ps
module Counter_tb;

    logic rstn, en , clk;
    logic [3:0] Q;

    top dut(
        .clk(clk),
        .rstn(rstn),
        .en(en),
        .Q(Q)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rstn = 0;
        en = 0;

        #3 rstn = 1;

        #7 en = 1;

        #160;

        en = 0;
        #30;

        en = 1;
        #20;
        rstn = 0;
        #10;
        rstn = 1;

        #50 $finish;
    end
endmodule
```

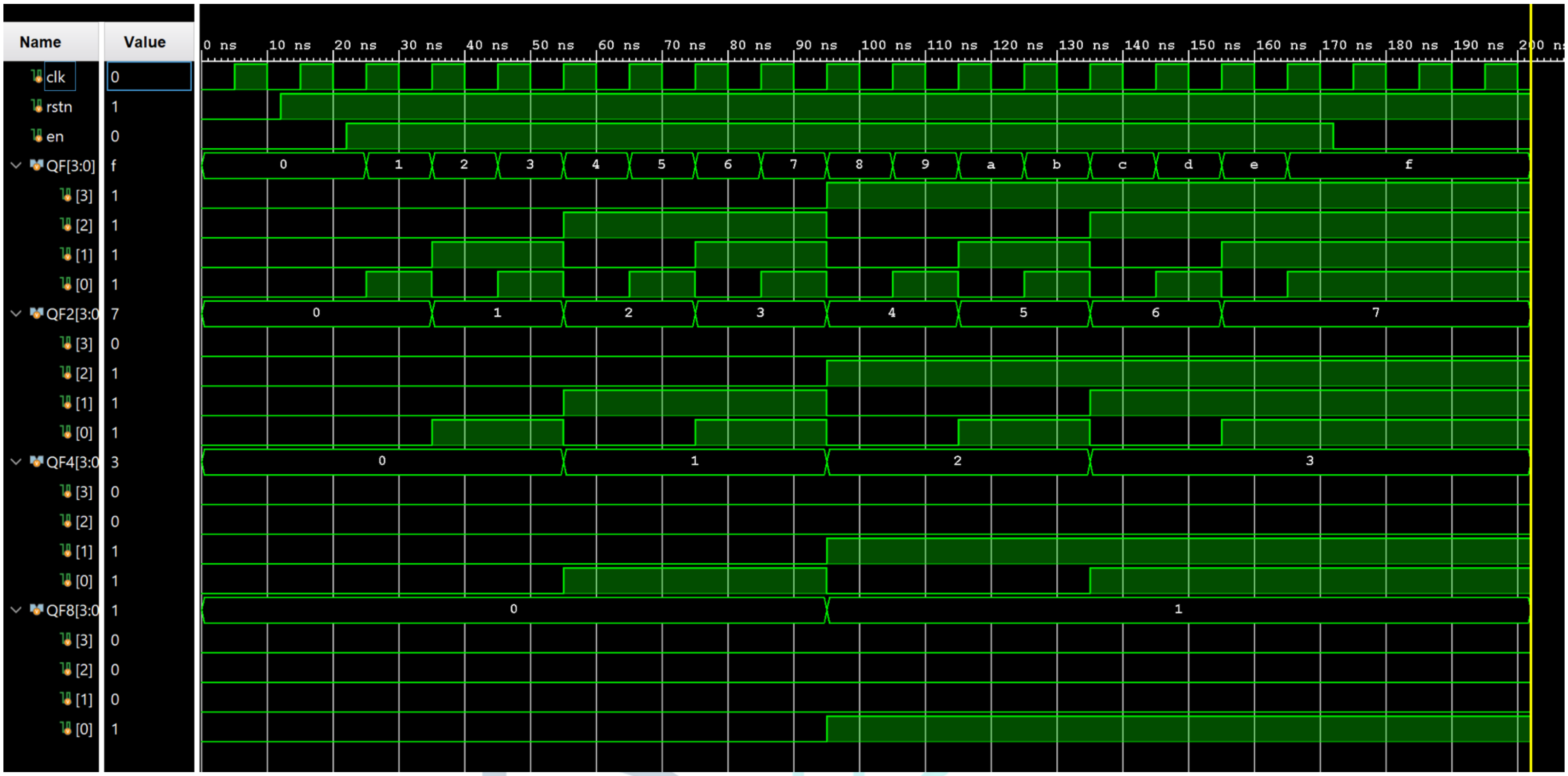
Task #1.3

```
module top_design (
    input  clk, rstn, en,
    output  [3:0] QF, QF2, QF4, QF8
    );
    logic clk_div2, clk_div4, clk_div8;

    clockDivider clk_div_inst (clk, rstn, clk_div2, clk_div4, clk_div8);
    top CounterF (clk, rstn, en, QF);
    top CounterF2 (clk_div2, rstn, en, QF2);
    top CounterF4 (clk_div4, rstn, en, QF4);
    top CounterF8 (clk_div8, rstn, en, QF8);

endmodule
```






```
`timescale 1ns / 1ps
module tb_top();
    logic clk, rstn, en;
    logic [3:0] QF, QF2, QF4, QF8;

    top_design dut (clk, rstn, en, QF, QF2, QF4, QF8);

    always #5 clk = ~clk;

    initial begin
        clk = 0; rstn = 0; en = 0;
        #12 rstn = 1;
        #10 en = 1;
        #150;
        en = 0;
        #30 $finish;
    end
endmodule
```


Name
Value
0 ns
10 ns
20 ns
30 ns
40 ns
50 ns
60 ns
70 ns
80 ns
90 ns
100 ns
110 ns
120 ns
130 ns
140 ns
150 ns
160 ns
170 ns
180 ns
190 ns
clk 0
rstn 1
en 0
QF[3:0] f
[3] 1
[2] 1
[1] 1
[0] 1
0 1 2 3 4 5 6 7 8 9 a b c d e f
QF2[3:0 7
[3] 0
[2] 1
[1] 1
[0] 1
0 1 2 3 4 5 6 7
QF4[3:0 3
[3] 0
[2] 0
[1] 1
[0] 1
0 1 2 3
QF8[3:0 1
[3] 0
[2] 0
[1] 0
[0] 1
0 1

Work distribution

Section	Farah&Joud	Tala&Horiah	Rana&Hoor&Sandra
Report	✓	✓	✓
Task 1.1	✓		
Task 1.2		✓	
Task 1.3			✓